

Noise Functions in Procedural Ski Resort Generation

Mia Kan

Advisor: Norman Badler, Adam Mally

University of Pennsylvania

ABSTRACT

Taking inspiration from “G.N.A.R.” the ski movie, the project aims to demonstrate the use of noise functions in procedurally generated terrain with a minimalistic obstacle avoidance game. Through Perlin noise and fractal brownian motion, the project investigates the algorithmic creation of ski slope surfaces, tree coverage, as well as simple avalanche simulation. Its intended purpose is to combine the playfulness in the culture of extreme snowsports with concepts in computer graphics.

G.N.A.R. the ski movie: <https://unofficialnetworks.com/gnar-the-movie/>

Git Repo: <https://github.com/phoenixviking/gnar>

1. INTRODUCTION

This project deals with procedural terrain generation, a principal component of computer graphics involving environment creation. It offers immersive experiences by creating vast and diverse landscapes or assets on-the-go, allowing for minimal data storage. Since it is able to achieve realistic simulations of worlds and behaviors, it enhances user experience as well as performance maximization. This senior project will use noise functions in order to generate a snow mountain game environment.

1.1 Design Goals

Procedural ski mountain generation is likely to appeal most to users familiar with snowsports as noise functions are already heavily used in popular games of today. Despite the specific target audience, this is intended to be an exploration in environment creation that can be of interest to any user that holds curiosity in algorithmic terrain generation, whether visually or technically.

1.2 Projects Proposed Features and Functionality

The below section will outline the proposed features and functionality of the project:

Feature 1 - Procedural Ski Mountain Generation

Feature 2 - Procedural Tree Generation

Feature 3 - Procedural Avalanche Generation

2. RELATED WORK

2.1 Procedural Content Generation

Procedural Content Generation (PCG) is utilized in different domains, notably in video games, simulation scenarios, and film production. In video games, PCG can automate the generation of levels, assets, and landscapes, enabling infinite gameplay possibilities. PCG also finds utility in simulating real-world scenarios for testing

purposes, such as robot control algorithms or risk assessment in natural floods [TC16]. Furthermore, it has shown increasing presence in film production to generate large quantities of similar objects or characters realistically. Examples such as Minecraft illustrate how PCG enhances user experiences by providing endless variations in immersive worlds. Overall, a versatile tool with diverse applications, PCG proves useful from entertainment purposes to practical simulation and analyses [TC16].

In the particular case of this project, procedural terrain generation refers to algorithms that will determine a height, color or asset presence at a particular 2D location, mapping some coordinate (x, y) to a z value, RGB value, or a boolean value through a mathematical function known as noise functions [Uni21]. Noise functions eliminate the need for visual assets as it is computed on the fly, and will reduce memory costs. It is useful in mimicking environment variation and maximizing performance.

2.2 Perlin Noise

Perlin noise, invented by Ken Perlin, is a popular method in computer graphics for generating random noise, and a mathematical function with useful traits including:

- A repeatable, band-limited pseudorandom function
- A bounded range [-1, 1]
- Continuous variation, no discontinuity
- Regions of relative stability
- Ability to combine with other noise functions [Cam18]

Perlin noise is computed through the following steps:

- Generate a finite sample of random values
- Generate a noise function that interpolates smoothly between these values
- Sum together various octaves of this function by scaling it down by factors of 1/2, and then applying a dampening value to each successive octave, so that high frequency variations are reduced [ME18]

In manipulation of Perlin noise, customization is possible over the below attributes:

- Amplitude
- Frequency
- Number of octaves
- Persistence [Uni21]

Seen in Figures 1 and 2 are texture maps of 2D Perlin noise functions where color intensities from (0, 0, 0) to (255, 255, 255) were determined by the output of the function.



Figure 1: *Texture map of Perlin noise* [Uni21]



Figure 2: *Texture map of Perlin noise, parameters adjusted* [Uni21]

2.3 Fractal Brownian Motion

Fractal Brownian Motion (fBm) is a technique that is often utilized for procedural landscape construction [GL14]. fBMs are usually constructed by creating smooth but deterministic randomness through a noise function of choice, and then constructing self-similarity with it [Qui19]. It is done by starting with a simple noise signal, then continuously adding increasingly smaller and detailed noise invocations. An example of a fBm texture map is seen in Figure 3.

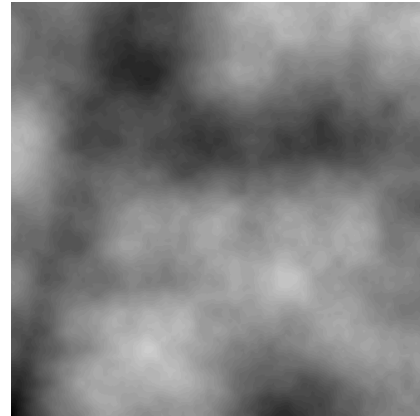


Figure 3: *Texture map of fBm* [GL14]

Many natural structures can be decomposed into a few big shapes that describe the overall contour, then a greater number of medium-sized shapes added on to distort the basic surface of the initial big shape, and finally, even more smaller shapes that supplement the detail on the contour of the previous larger shapes [Qui19]. For this reason, fBm is widely used for mountains, clouds, bark textures, and many environmental features in video games and cinema.

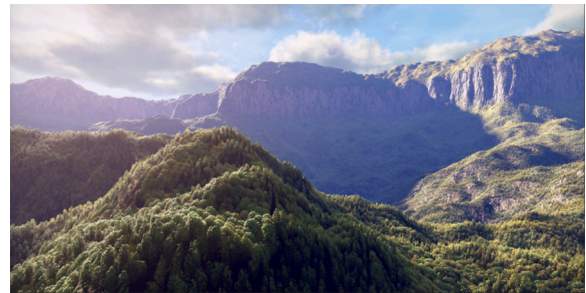


Figure 4: *Terrain, clouds, the tree distribution, tree color variations, and canopy details implemented with fBm. "Rainforest", 2016:*
<https://www.shadertoy.com/view/4ttSWf> [Qui14]

3. PROJECT PROPOSAL

This senior project will aim to show the application of noise functions in mountain, tree and avalanche generation through a simple game, titled GNAR (pronounced "narr").

3.1 Anticipated Approach

The game will take inspiration from "G.N.A.R.", a ski movie which celebrates the playful spirit of skiing through a unique blend of comedy, athleticism, and camaraderie. The film follows a collection of skiers navigating various challenges and adventures on a range of resorts while adhering to the "G.N.A.R." (Gaffney's Numerical Assessment of Radness) point system. Throughout the movie, the skiers compete to outdo each other in daring stunts and creative antics, all while plummeting down terrain of extreme difficulty, including but not limited to cliffs, jumps, and tree runs.

To capture the thrilling spirit of skiing enthusiasts, the player will be a skier descending down a procedurally generated ski mountain, running from an avalanche at their tail and dodging trees as obstacles. The avalanche animation, slope texture and tree patches will be procedurally generated using the noise functions outlined in section 2.

The user can control the player's movement with their mouse, allowing them to move left and right and dodge the trees. The skier will descend the slope at a constant rate, and therefore the game will feature a scrolling screen that frames the player from above at a fixed screen position. The terrain in the background will move and update its bounding box relative to the skier, allowing the screen to display the procedurally generated slope and trees as the skier changes position.

The player is rewarded with G.N.A.R points as they plummet down the slope. Their points will increase incrementally as they manage to continue avoiding trees, and upon colliding with a tree patch, the game is over and they will be prompted to restart or quit the game.

To embody the playful nature of the game of G.N.A.R, the rendering will be implemented using a pixelated aesthetic which will feature pixel squares as the building blocks of the screen space. The simplicity will also aim to enhance the accessibility of the game.

3.2 Target Platforms

The project will be built on pygame, a game development library using Python.

3.3 Evaluation Criteria

- Realistic aesthetics of a smooth, moderately bumpy slope surface
- Realistic aesthetics of tree patch shapes and placement
- Believable animation of the avalanche powder cloud behind the player
- Smooth movement of the player in the x and y direction
- Minimalistic and enjoyable gameplay

4. PROJECT TIMELINE

Pre-Project (01/26/24)

- Idea Brainstorms

Project Initialization (02/09/24)

- Completed background research on noise functions
- Project Proposal
- Decision on game over condition
- Decision on framework/platform

Alpha Review (02/23/24)

- Scrolling background terrain
- Player controls
- Implemented noise functions
- Procedurally generated slope, trees, avalanche

Beta Review (03/25/24)

- Ski tracks behind player's skis
- Game states implemented
- Updated documentation

Project Final Deliverables (05/10/24)

- Pixel aesthetics polished - colors, spacing, game state assets
- Project Specification Document complete
- Demo Video

Project Future Tasks

- Adding collectable GNAR tasks on slope, allocates extra reward points to player
- Selection of terrain, determined by player choice of ski resort

Figure 5 shows a gantt chart displaying project tasks against an intended timeline.

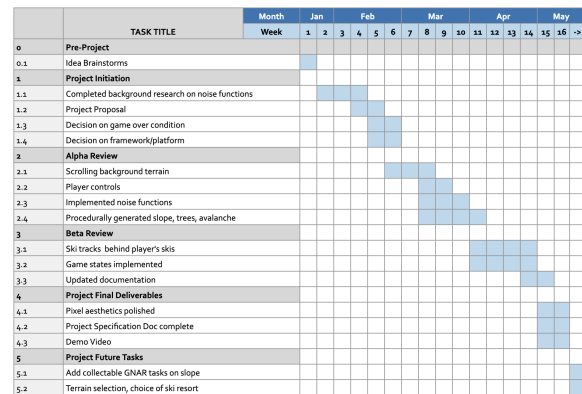


Figure 5: Project Gantt chart

5. Method

1. Running pygame window

First, the basic game infrastructure was implemented by creating skeletal files with the pygame library. The mechanism for a game window was set up (Figure 6). Refer to Appendix A for relevant code excerpt.



Figure 6: Pygame game window

2. Scrolling screen

The screen rendering framework, as well as the player-terrain interaction was then implemented to ensure a functioning scrolling screen (Figure 7). It also confirmed

the user's ability for x-axis movement in addition to the y-axis descent. Refer to Appendix B and C for relevant code excerpts.

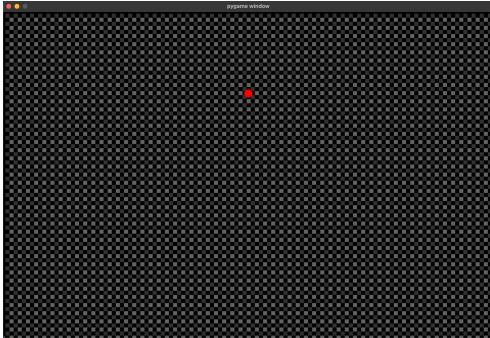


Figure 7: Snapshot of initial stages of a scrolling background terrain dictated by the player in red

3. Noise functions and texture map

The Noise class was implemented with the perlin and fBm noise functions. Furthermore, the pixel-block rendering algorithm was implemented to enable visibility of the working terrain generations as shown in Figure 8. Refer to Appendix D and H for relevant code excerpts.

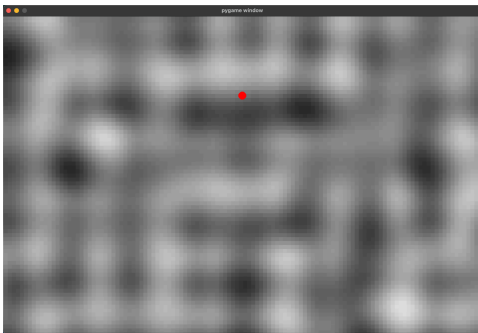


Figure 8: Perlin texture map rendered in pixel blocks

5. Experimentation with Slope and Obstacle Generation

Significant time was spent on trial and error with Perlin and fBm noise combinations, individual function parameters and scaling noise functions to display the optimal aesthetics for an abstracted pixel look that was still realistic for ski resort terrain. Tweaked parameters included but were not limited to interpolation weights, RGB ranges, and cutoff values.

The slope texture is generated by creating mellow bumps through largely scaled then blurred fBm noise, while tree obstacles are generated through a mix of both Perlin and fBm noise. The perlin noise creates 'islands' or 'patches' of trees, while the fBm helps such patches look less regular and more random or rough at the edges and in shape. See Figure 9 for visual representation; refer to Appendix D, E, F, G for relevant code excerpts.

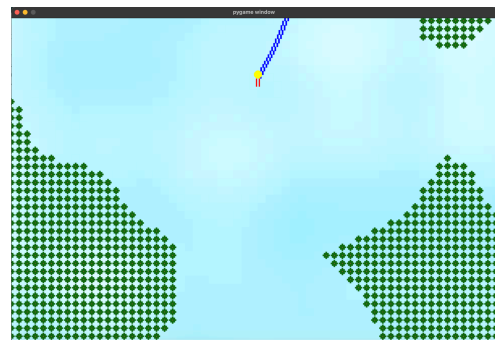


Figure 9: Working stages of tree generation and slope surface generation

7. Avalanche Animation

The avalanche was added by reusing fBm noise – given an x value along a fixed line on the y-axis spanning from the left edge of the right edge of the screen, the fBm output was used to determine the bottom coordinate of the avalanche debris in some given y-range to animate a falling avalanche in the top chunk of the screen. The bottom pixel block was darkened in color to a light gray to enhance the aesthetics, reflected in Figure 10. Refer to Appendix I for relevant code excerpts.

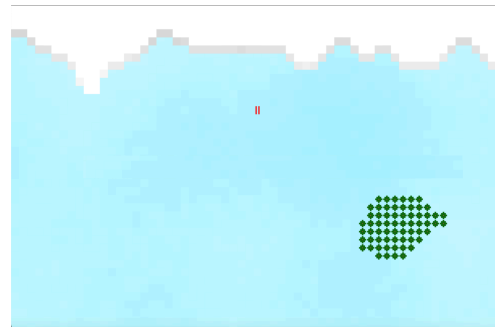


Figure 10: Snapshot of avalanche animation

6. Ski tracks

A visual detail addition, seen in Figure 11; between each translation of the skier, the Player class keeps track of a certain number of locations where the back tips of the player's skis passed between the current position and the new position. It creates a more grounded effect of the player plummeting down the slope. It also aims to amplify the thrill of the avalanche as the tracks disappear under the snow debris. Refer to Appendix J for relevant code excerpt.



Figure 11: *Light gray ski tracks*

8. Game States

Game controls for start, pause, and quit were implemented as seen in figures 12-15. A scoreboard for “GNAR points” was also added. Refer to Appendix K, L, M for relevant code excerpts.



Figure 12: *Startup menu state*



Figure 13: *Paused state*

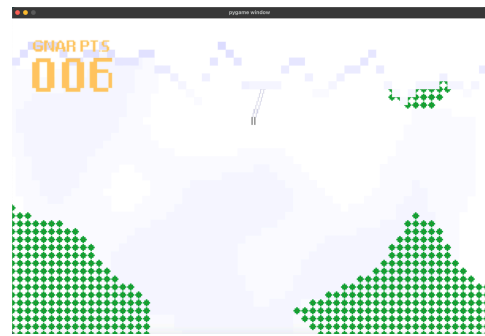


Figure 14: *Playing state*



Figure 15: *Game over state*

6. RESULTS

In this section the project implementation will be compared against the evaluation criteria in section 3.3.

A) Realistic aesthetics of a smooth, moderately bumpy slope surface



Figure 16: *Ski resort terrain [Pow-]*



Figure 17: *Subtle terrain variability in procedurally generated surface*

In a ski resort similar to that in Figure 16, a long, groomed flat surface can be observed in tandem with ungroomed hills and inclines of greater range. Although a mellow, bumpy slope surface was achieved in this project using perlin noise shown in Figure 17, it is very subtle in color change and the game therefore could have benefitted from a more obvious indication of variable ski terrain.

However, the constraints of the player descending in constant velocity in combination with the top down camera view posed an obstacle in rendering a slope with a greater range of height. A greater range of height would have demanded a physics-based velocity computation of the player's descent to uphold the standards of realism of the game and to not impose unease on the user's experience of gameplay controls.

B) Realistic aesthetics of tree patch shapes and placement



Figure 18: *Aerial view of Whistler Blackcomb [Fai23]*

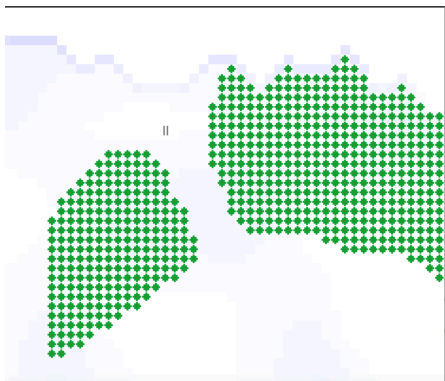


Figure 19: *Procedurally generated tree patch*

The pixelated aesthetics of the game make the patches of trees believable enough on the assumption that the environment was based on a completely ungroomed mountain (Figure 19). On the other hand, based on aerial images of ski resorts as seen in Figure 18, perhaps a completely different noise function would have proved more applicable in order to produce a higher density of trees, as well as a number of carved out “runs” that feature straighter, descending columns of no trees.

C) Believable animation of the avalanche powder cloud behind the player



Figure 20: *Powder cloud of an avalanche [SB22]*



Figure 21: *Snapshot of avalanche animation*

Based on the comparison between Figure 20 and 21, and factoring in the error margin given by the pixelated game aesthetic, the avalanche simulation appears realistic.

D) Smooth movement of the player in the x and y direction

Movement of the player is smooth in both the x and y direction.

E) Minimalistic & enjoyable gameplay

Although a subjective criteria, the game is minimalistic given the user is only required to manipulate the mouse and the only threat posed are the trees. Improvements can be suggested in producing more reasons for the player to feel rewarded or threatened to keep the game engaging, such as more opportunities for points or more types of obstacles.

7. CONCLUSIONS and FUTURE WORK

In conclusion, this project was successful in utilizing two popular procedural content generation methods, Perlin and fBm noise functions, in order to simulate a simplistic ski resort. It was able to demonstrate that noise functions don't necessarily require complex applications as often seen in 3D games and cinematography.

In future iterations, this project can take from noise function optimization in order to fix slight lag issues with the rendering of pixel blocks. In addition, further research on a wider array of applicable noise functions, or alternatively, greater time spent on noise function parameter adjustments is encouraged in order to bring the

terrain generation closer to realism. To enhance the playful spirit of the game, terrain choice and customization across different types of ski resorts such as icy European Alps / powdery Japanese mountains / Whistler tree runs may be a great touch.

APPENDIX

A. Pygame framework initialization

```
class SkiPatrol:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((1280, 720))
        self.clock = pygame.time.Clock()
        self.running = True

    # game loop
    def play(self):
        while self.running:
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    self.running = False

            # do the game logic
            # fill the screen
            self.screen.fill("white")
            # drawings here
            pygame.display.flip()

        pygame.quit()
```

B. Scrolling terrain in relation to player y-translation

```
# limit FPS to 60
self.dt = self.clock.tick(60) / 1000
self.player.set_pos_y(self.player.get_pos_y() + 20 * self.dt)
self.slope.update_screen_UL_pos()
```

C. Scrolling and player in x-direction through mouse

```
# handle events
def handle_events(self):
    mouse_pos = pygame.mouse.get_pos()
    pos_x_delta = mouse_pos[0] - self.player.get_pos_x()
    self.player.set_pos_x(self.player.get_pos_x() +
                          (0.08 * pos_x_delta) * self.dt)
    self.slope.update_screen_UL_pos()
```

D. Perlin noise umbrella function

```
@staticmethod
# divide uv by the amount you want to scale the perlin noise!
def perlin(uv):
    uvXLYL = Noise.floor(uv)
    uvXHLYL = uvXLYL + pygame.Vector2(1, 0)
    uvXHYYH = uvXLYL + pygame.Vector2(1, 1)
    uvXLYYH = uvXLYL + pygame.Vector2(0, 1)

    surflet_sum = (Noise.surflet(uv, uvXLYL) + Noise.surflet(uv, uvXHLYL)
                  + Noise.surflet(uv, uvXHYYH) + Noise.surflet(uv, uvXLYYH))

    # set output from 0 to 1
    sum_normalized = (surflet_sum + 1.0) / 2.0

    return sum_normalized
```

E. fBm noise umbrella function

```
@staticmethod
# fractal brownian motion
def fbm(uv):
    total = 0
    persistence = 0.15
    octaves = 10 # changes frequency of ripples
    freq = 18 # changes spacing of noise
    amp = 3.5 # changes height of spikes
    for i in range(octaves):
        freq *= 1.8 # was 2.0
        amp *= persistence
        total += Noise.interp_noise(uv.x * freq, uv.y * freq) * amp

    return pow(total, 1.4)
```

F. Procedural slope generation

```
@staticmethod
# outputs a decimal in range [0, 1]
def slope_noise(pos, fbm_scale):
    # get fbm noise
    fbm = Noise.fbm(pos / fbm_scale)
    # fbm seems to output in range 0.3 to 0.7, so scale it over
    # maximum spread in [0, 1]
    scaled = min(((fbm / 0.4) + 0.3), 1.0)
    # divide the noise into 0.2 intervals to give a more pixelated look
    slope_noise = min(0.2 * (scaled // 0.2) + 0.2, 1.0)

    return slope_noise
```

G. Procedural tree generation

```
@staticmethod
# outputs a decimal in range [0, 1]
def trees_noise(pos, perlin_scale, fbm_scale):
    # get perlin noise (creates patches)
    perlin = Noise.perlin(pos / perlin_scale)
    # get fbm noise (creates irregularity)
    fbm = Noise.fbm(pos / fbm_scale)
    # LERP between perlin and fbm
    trees_noise = Noise.mix(fbm, perlin, a=0.65)

    return trees_noise
```

H. Slope rendering via pixel block divisions

```
# draw the block based on slope height
def draw_slope_block(self, screen_x, screen_y, height):
    # decide the shading of the block
    # r_g_b = 150 * height + 105
    # paint the slope an icy-light blue-type color
    # https://imagecolorpicker.com/color-code/e4f7ff
    r = 18 * height + 237
    g = 13 * height + 242
    color = (r, g, 255)
    rect_w = Slope.block_size
    rect_h = Slope.block_size

    rect = pygame.Rect(screen_x, screen_y, rect_w, rect_h)
    pygame.draw.rect(self.screen, color, rect)
```

I. Avalanche animation

```
# draw the avalanche as an array of long snow columns that are of
# dimension Slope.block_size * avalanche_length
for x in range(0, self.screen_w, Slope.block_size):
    # want a noisy avalanche, clip noise from 0 to 1
    pos = self.get_screen_UL_pos() + pygame.Vector2(x, 0)
    noise = min(((Noise.avalanche_noise(pos, fbm_scale: 2000)) / 0.45) + 0.1, 1.0)
    avalanche_height = (avalanche_range * noise) + avalanche_min

    # avalanche column
    avalanche_length = (Slope.block_size * (avalanche_height // Slope.block_size)
                        - Slope.block_size)
    avalanche_column = pygame.Rect(x, 0, Slope.block_size, avalanche_length)
    # last block of the avalanche column will be a grey color
    debris_hue = 40 * noise + 215
    debris_color = (debris_hue, debris_hue, 255)
    avalanche_debris = pygame.Rect(x, avalanche_length,
                                   Slope.block_size, Slope.block_size)

    # draw white avalanche with the shadow
    pygame.draw.rect(self.screen, white, avalanche_column)
    pygame.draw.rect(self.screen, debris_color, avalanche_debris)
```

J. Ski tracks computation upon player translation

```
# translate player
def translate(self, delta_x, delta_y):
    # delta y is always negative
    y_inc = -1
    x_inc = 2
    new_pos = self.pos + pygame.Vector2(delta_x, delta_y)
    if delta_x < 0:
        x_inc = -2

    # add a track for every (+x, -1) or (-x, -1) movement from curr pos to new pos
    for y in range(int(self.pos.y), int(new_pos.y), y_inc):
        # use floor + ceil to make sure that tracks are still being added when
        # the mouse pos is directly under the player
        for x in range(int(math.floor(self.pos.x)), int(math.ceil(new_pos.x)), x_inc):
            fresh_track = pygame.Vector2(x, y)
            self.add_track(fresh_track)

    self.pos.x += delta_x
    self.pos.y += delta_y
```

K. Handling key press events for game states

```
# handle key press events
def handle_events(self):
    # respond to arrow key events
    keys = pygame.key.get_pressed()
    if keys[pygame.K_RETURN]:
        if self.pause:
            self.pause = False
        else:
            if self.game_over:
                self.game_over = False
                self.player.reset()

            self.ticks = 0
            self.start = True
    if keys[pygame.K_ESCAPE]:
        if self.pause:
            self.running = False
        else:
            self.pause = True
```

L. Game over condition

```
# draw some score from 000-999
def draw_score(self):
    self.screen.blit(self.gnar_pts_img, self.gnar_pts_img.get_rect(center=(150, 70)))

    # print each digit
    hundreds_digit = math.floor(self.ticks / 100)
    tens_digit = math.floor((self.ticks - (100 * hundreds_digit)) / 10)
    ones_digit = math.floor(self.ticks - (100 * hundreds_digit) - (10 * tens_digit))
    hundreds_img = self.score_0
    tens_img = self.score_0
    ones_img = self.score_0

    # get corresponding digit assets
    match hundreds_digit:
        case 0:
            hundreds_img = self.score_0
```

M. Scoreboard computation

```
# wait to begin the game
if not self.start:
    # draw the start banner
    self.draw_start()
    pygame.display.flip()
    continue

# set FPS to 40
self.dt = self.clock.tick(40) / 1000

# check if game over
if self.slope.player_collided():
    # draw game over banner
    self.draw_game_over()
    self.game_over = True
else:
```

References

- [Cam18] Cambridge University. (2018). Appendix B: Perlin Noise. Department of Computer Science and Technology. <https://www.cl.cam.ac.uk/teaching/1718/FGraphics/Appendix%20B%20-%20Perlin%20Noise.pdf>
- [Fai23] Fairmont. (2023). [Photograph]. Home Fairmont. <https://www.fairmont.com/whistler/ski/skii-ng-in-whistler/>
- [GL14] Gonzalez Vivo, P., & Lowe, J. (2014). Fractal Brownian Motion. The Book of Shaders. <https://thebookofshaders.com/13/>
- [ME18] Mount, D., & Eastman, R. (2018). CMSC 425: Lecture 13 - Procedural Generation: 2D Perlin Noise. UMD Department of Computer Science. <https://www.cs.umd.edu/class/spring2018/cmssc425/Lects/lect13-2d-perlin.pdf>
- [Pow–] Powderhounds. (n.d.). Ski Whistler BC [Photograph]. Powderhounds. <https://www.powderhounds.com/Canada/Whistler-Blackcomb/Ski-Resort.aspx>
- [Qui19] Quilez, I. (2019). fBm. Inigo Quilez :: computer graphics, mathematics, shaders, fractals, demoscene and more. <https://iquilezles.org/articles/fbm/>
- [SB22] SBG San Antonio Staff Reports. (2022, March 21). [Photograph]. CBS Austin. <https://cbsaustin.com/news/nation-world/dog-presumed-dead-after-avalanche-found-alive-days-later-snow-winter-mountains-ski-snowboarding-buried-rescue>
- [TC16] Tomassetti, F., & Curtis, B. (2016, March 28). Diving into procedural content generation, with WorldEngine — Smashing magazine. Smashing Magazine. <https://www.smashingmagazine.com/2016/03/procedural-content-generation-introduction/>

- [Uni21] University of Texas. (2021). Noise Functions. Department of Computer Science.
<https://www.cs.utexas.edu/~theshark/courses/cs354/lectures/cs354-21.pdf>*Graphics Forum* 9, 4 (Dec. 1990), 371–372.